

Amirkabir University of Technology
(Tehran Polytechnic)

Instructor:

Dr. Mohamed Khalooei.

Student Name: Ahmed Kamil Ahmed

404131074

Yes/No Audio Classification Using Machine Learning

1. Introduction

Speech processing is an important area in computer science. It helps computers understand and analyze human speech. Today, speech processing is used in many applications, such as voice assistants, speech recognition systems, and voice-controlled devices.

Audio classification is one of the tasks in speech processing. The main idea is to analyze an audio signal and classify it into a specific category. For example, a system can identify whether a person says "Yes" or "No" by analyzing the speech signal.

2. Dataset Description

In this task, I used the YESNO dataset from OpenSLR. This dataset contains recordings of a person saying the words "Yes" and "No" in different sequences.

The dataset contains 60 audio recordings in WAV format. Each recording includes eight spoken words, which can be either Yes or No.

The audio files are named using a sequence of zeros and ones. Each number represents one spoken word:

- **0** represents "No".
- **1** represents "Yes".

For example, a file named 1_0_1_0_1_0_1_0.wav represents the sequence: Yes, No, Yes, No, Yes, No, Yes, No.

2.1 Loading the Audio Data

I downloaded the dataset and loaded the audio files using Python and the Librosa library in Google Colab.

After loading the recordings, I examined the audio signals and checked their sample rates and durations.

The sample rate tells us how many audio samples are recorded per second. This information is important because it helps us analyze the audio signal correctly.

2.2 Audio Segmentation

Since each recording contains eight spoken words, I used a silence-based method to divide the recordings into smaller audio segments.

The method detects parts of the recording that contain sound and separates them from silent parts.

During this process, 41 recordings were divided into exactly eight segments. The remaining 19 recordings were excluded because the method detected a different number of segments.

As a result, I obtained 328 audio segments for the classification experiment.

2.3 Dataset Summary

Description	Value
Dataset Name	YESNO
Source	OpenSLR
Audio Format	WAV
Total Recordings	60
Words per Recording	8
Classification Classes	Yes (1), No (0)
Accepted Recordings	41
Excluded Recordings	19
Extr	

3. Audio Preprocessing and Visualization

Before training the machine learning models, I analyzed the audio recordings to understand their structure and prepare them for classification.

I used Python and the Librosa library in Google Colab to load, process, and visualize the audio signals.

3.1 Audio Signal Analysis

First, I selected an audio file from the YESNO dataset and loaded it using Librosa.

I checked the sample rate and duration of the recording. I also listened to the audio to understand how the Yes and No words were spoken.

This step helped me understand the audio data before extracting features.

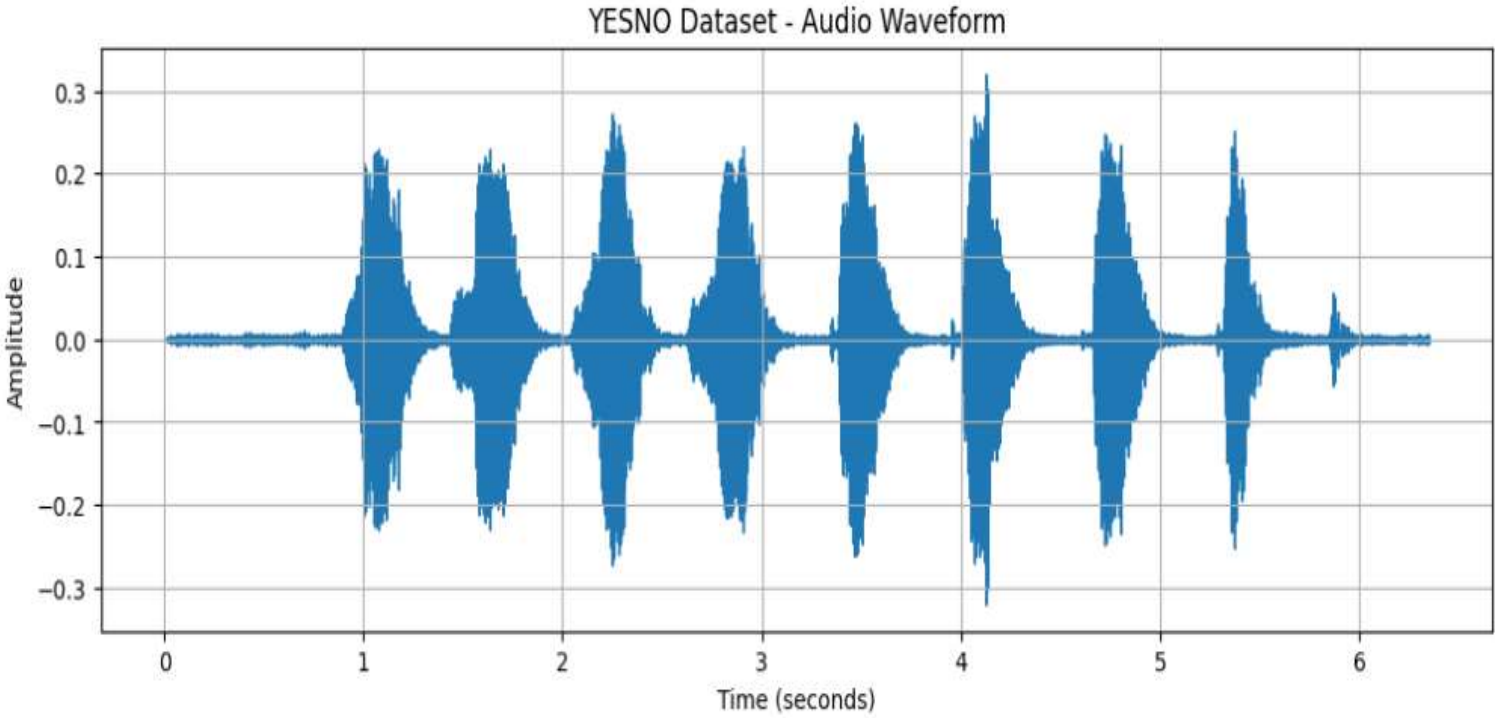
3.2 Waveform Visualization

I plotted the waveform of the selected audio recording.

A waveform shows how the amplitude of an audio signal changes over time. The horizontal axis represents time in seconds, while the vertical axis represents amplitude.

The waveform helped me observe the speech parts and the silent parts of the recording.

Figure 1. Waveform of a YESNO audio recording



3.3 Spectrogram Visualization

Next, I generated a spectrogram to examine the frequency information of the audio signal.

A spectrogram shows how the frequency content of a signal changes over time.

The horizontal axis represents time, the vertical axis represents frequency, and the colors represent the strength of the signal at different frequencies.

This visualization helped me understand the frequency patterns of the spoken words.

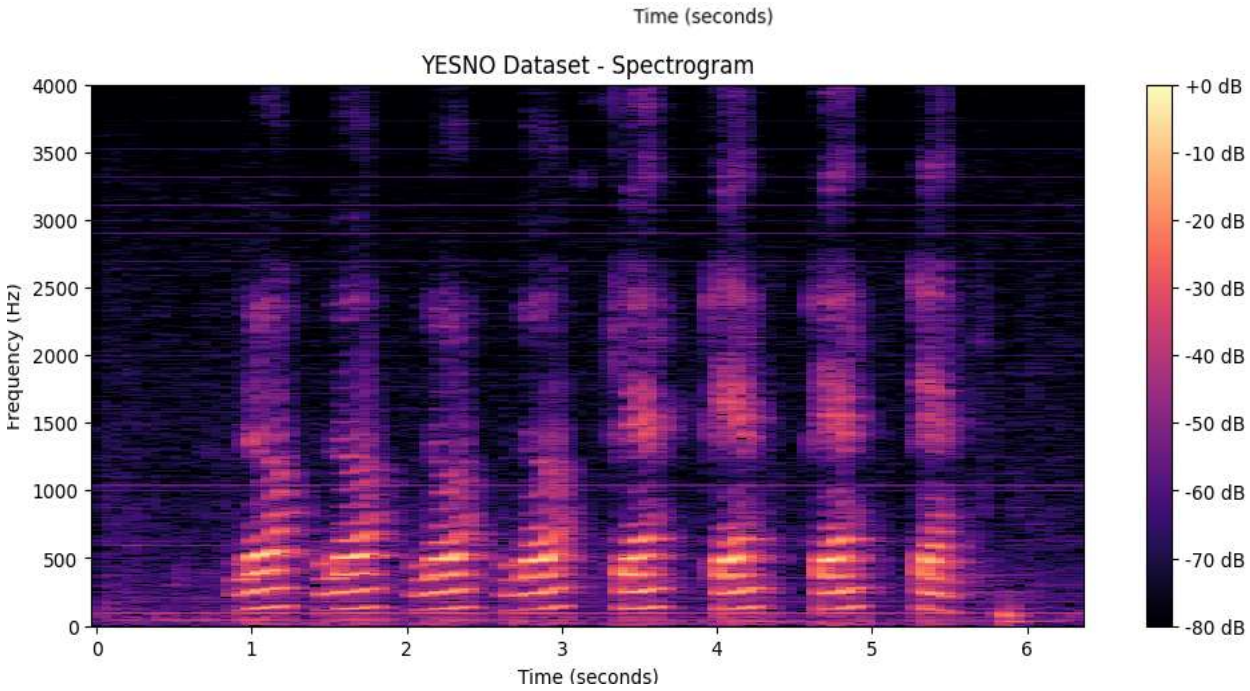


Figure 2: Spectrogram of a YESNO audio recording.

4. Feature Extraction Using MFCC

Feature extraction is an important step in audio classification. Instead of using the raw audio signal directly, we extract useful information that helps machine learning models recognize different sounds.

In this task, I used Mel-Frequency Cepstral Coefficients (MFCCs) to extract features from the Yes and No audio segments.

4.1 What is MFCC?

MFCC is a common method used in speech processing. It represents important characteristics of the speech signal, especially its frequency information.

MFCC features help machine learning models find differences between spoken words.

4.2 Extracting MFCC Features

After dividing the audio recordings into individual words, I extracted 13 MFCC coefficients from each speech segment using the Librosa library.

Since the number of audio frames can be different for each word, I calculated the mean and standard deviation of each MFCC coefficient.

This produced:

- 13 mean values.
- 13 standard deviation values.
- A total of 26 features for each audio segment.

These 26 features were used as input to the machine learning models.

4.3 MFCC Visualization

I also plotted the MFCC coefficients of a selected audio recording to see how the features change over time.

The horizontal axis represents time, while the vertical axis represents the MFCC coefficients.

The colors show the values of the coefficients at different times.

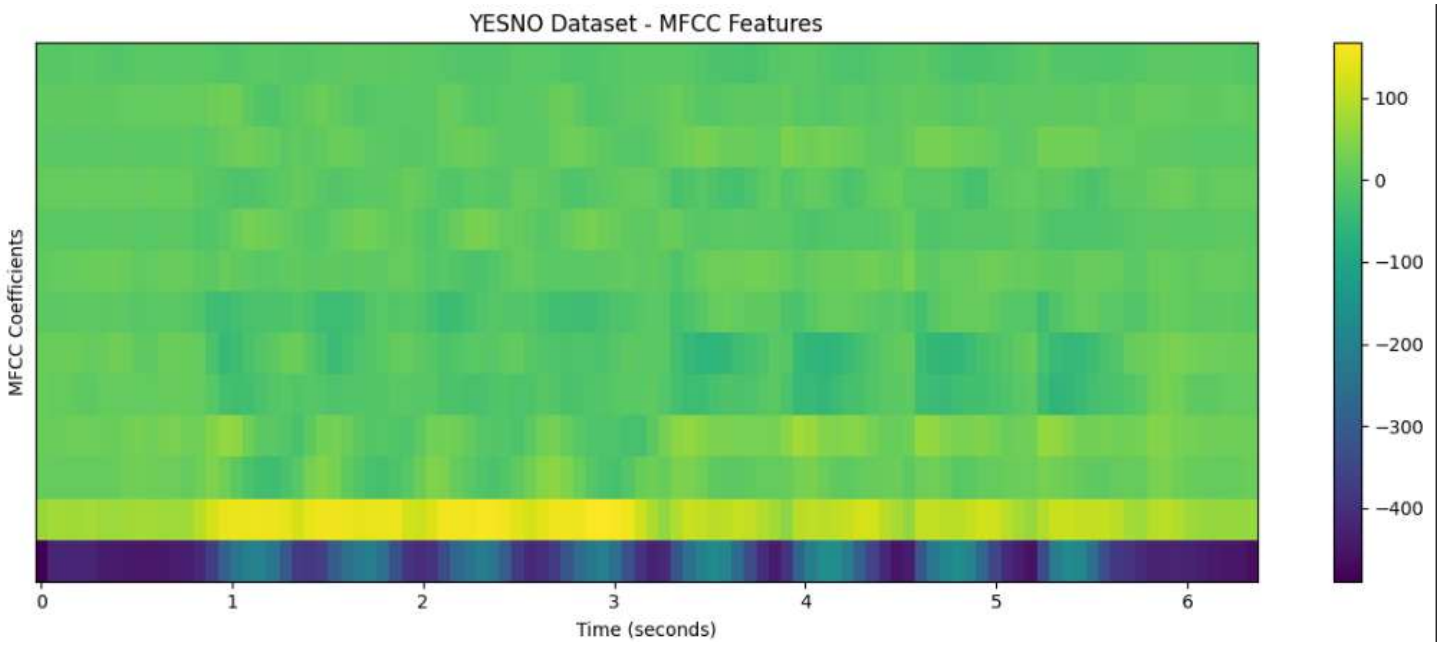


Figure 3: MFCC features of a YESNO audio recording.

4.4 Preparing Features for Classification

After extracting the MFCC features, I prepared the feature matrix and the corresponding labels.

Each row of the feature matrix represents one spoken word, and each row contains 26 numerical features.

The labels were taken from the audio filenames, where 0 represents No and 1 represents Yes.

The final dataset contained 328 speech segments:

- 176 Yes segments.
- 152 No segments.

These features and labels were used to train and test the classification models.

5. Machine Learning Classification Models

After extracting the MFCC features, I used four machine learning models to classify the spoken words into two classes: Yes and No.

The models were Support Vector Machine (SVM), Neural Network (MLP), Random Forest, and K-Nearest Neighbors (KNN).

I selected these models to compare different classification methods and find out how well they can recognize the two spoken words.

5.1 Support Vector Machine (SVM)

Support Vector Machine is a supervised machine learning algorithm used for classification.

SVM tries to find a boundary that separates different classes based on their features.

In this experiment, I used SVM with an RBF kernel to classify the MFCC features into Yes and No.

I also used StandardScaler to scale the features before training the model.

5.2 Neural Network (MLP)

A Neural Network is a machine learning model that learns patterns from data using connected artificial neurons.

In this task, I used a Multi-Layer Perceptron (MLP) with two hidden layers containing 64 and 32 neurons.

The model used the ReLU activation function and the Adam optimizer.

The goal of the Neural Network was to learn the differences between the MFCC features of Yes and No words.

5.3 Random Forest

Random Forest is a machine learning algorithm that uses multi

6. Experimental Setup and Evaluation

In this experiment, I used Google Colab and Python to train and test the four machine learning models.

After extracting the MFCC features, I divided the audio data into training and testing sets.

6.1 Training and Testing Data

I used the train_test_split() function from Scikit-learn to divide the original audio recordings into two groups:

- 75% for training.
- 25% for testing.

I used random_state = 42 to make the data split reproducible.

The recordings were divided before extracting the individual word segments. This helped prevent words from the same recording from appearing in both the training and testing sets.

After applying the segmentation method and keeping recordings with exactly eight detected words, I obtained the following data:

Dataset	Number of Segments
Training Data	248
Testing Data	80
Total	328

6.2 Training the Models

I trained four classification models using the training data:

1. Support Vector Machine (SVM)
2. Neural Network (MLP)
3. Random Forest
4. K-Nearest Neighbors (KNN)

Each model used the same 26 MFCC-based features as input.

For SVM, Neural Network, and KNN, I used StandardScaler to normalize the features.

The scaling process was included in the training pipeline to avoid using information from the test data during training.

6.3 Model Evaluation

After training the models, I tested them using 80 audio segments that were not used during training.

I used classification accuracy to measure the performance of each model.

Accuracy measures the percentage of correctly classified samples.

The accuracy formula is:

Accuracy = (Number of Correct Predictions / Total Number of Predictions) × 100

For example, if a model correctly classifies 72 out of 80 audio segments:

Accuracy = (72 / 80) × 100 = 90%

I also used a confusion matrix to examine the predictions of the SVM model.

The confusion matrix shows the number of correct and incorrect predictions for each class.

6.4 Evaluation Goal

The main goal of the evaluation was to compare the performance of the four models using the same test data.

This comparison helped me understand how different machine learning algorithms perform on the Yes/No classification task.

7. Results and Discussion

After training the four machine learning models, I tested their performance using 80 speech segments from the YESNO dataset.

The test data contained 37 No segments and 43 Yes segments.

7.1 Classification Results

I evaluated the four models using classification accuracy. The results are shown in Table 3.

Table 3: Accuracy of the Four Classification Models

Classification Model	Test Accuracy
Support Vector Machine (SVM)	100%
Neural Network (MLP)	100%
Random Forest	100%
K-Nearest Neighbors (KNN)	100%

All four models achieved 100% accuracy on the selected test data.

This means that each model correctly classified all 80 test segments in this experiment.

7.2 Comparison of the Models

To compare the performance of the four models, I created a bar chart using Python.\

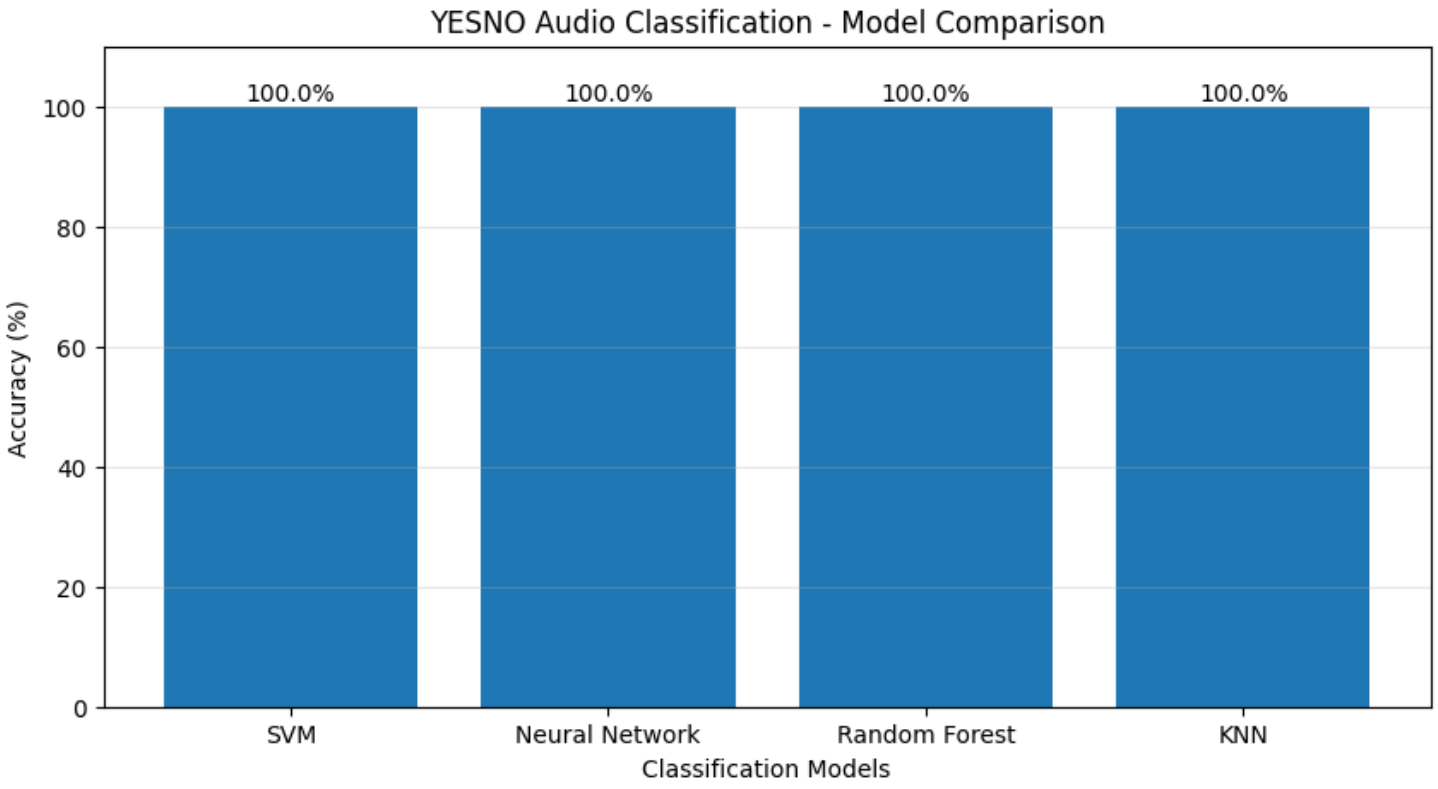


Figure 4: Accuracy comparison of the four classification models.

The chart shows that all four models achieved the same accuracy.

In this experiment, no model performed better than the others based on test accuracy.

However, these results do not mean that all four models will always have the same performance on different datasets.

7.3 Confusion Matrix

I also used a confusion matrix to examine the classification results of the SVM model.

Table 4: Confusion Matrix of the SVM Model

Actual / Predicted	No	Yes
No	37	0
Yes	0	43

The confusion matrix shows that the SVM model correctly classified 37 No segments and 43 Yes segments.

There were no incorrect predictions in the selected test set.\

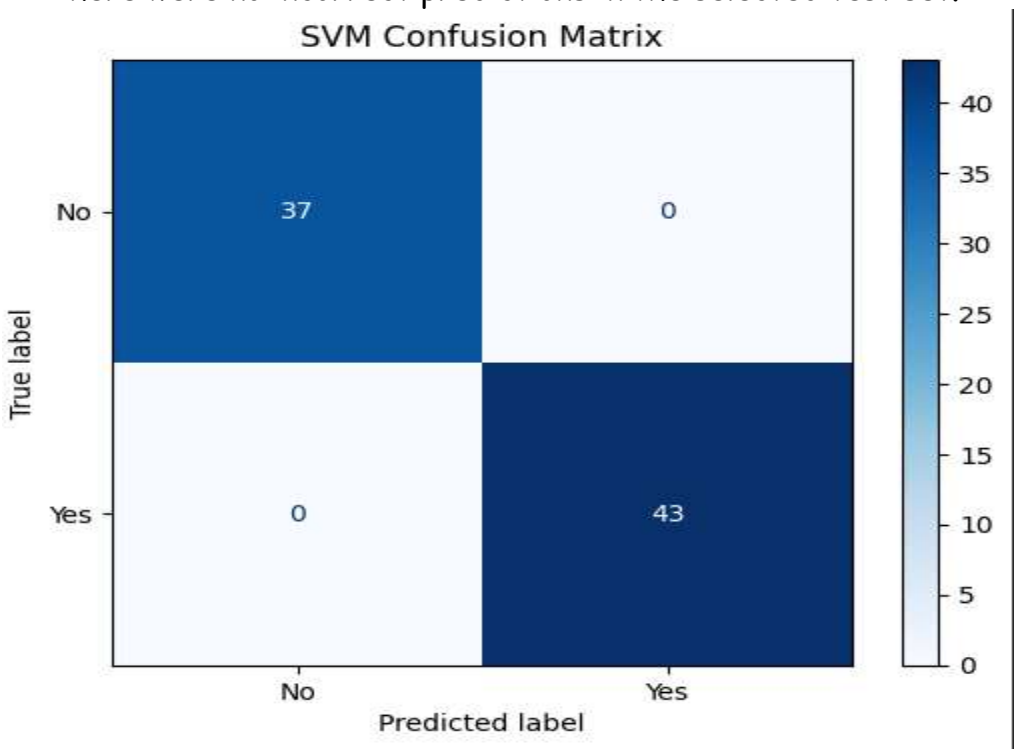


Figure 5: Confusion matrix of the SVM model.

7.4 Discussion

The results show that the four machine learning models performed well on the selected YESNO test data.

One possible reason for the high accuracy is that the task contains only two classes: Yes and No.

Also, the MFCC features may contain useful information that helps the models distinguish between these two spoken words.

However, I used a relatively small dataset, and 19 recordings were excluded because the segmentation method did not detect exactly eight speech segments.

Another important point is that the detected segments were assigned labels based on their order in the original filenames.

8. Limitations and Conclusion

8.1 Limitations

During this experiment, I faced some limitations.

First, the YESNO dataset contains only 60 audio recordings, which is a relatively small dataset.

Second, the audio segmentation method did not work correctly for all recordings. I used 41 recordings and excluded 19 recordings because they did not produce exactly eight speech segments.

Also, the experiment was limited to two classes: Yes and No. The models may give different results when tested with more complex speech data.

8.2 Conclusion

In this task, I developed a simple audio classification system using the YESNO dataset.

I used Librosa to process the audio recordings and extracted 26 MFCC-based features from each speech segment.

Then, I trained and tested four machine learning models: SVM, Neural Network, Random Forest, and KNN.

All four models achieved 100% accuracy on the selected test set of 80 speech segments.



From this experiment, I learned how to prepare audio data, extract MFCC features, train classification models, and compare their results.

In future work, I would like to improve the segmentation method and test the models using more audio recordings to check their performance.